

FARO — Security, protocols and data handling

Book 1 of 3 · For the institution, the IT team and the jury

Version: September 2026 · SHIFT 2026 Challenge · Gamification

What this book is for

A university that connects a tool to its Canvas is lending something valuable: access to its students' academic data. This book explains, plainly, what FARO does with that access, which protocols it uses, why those protocols are secure, and which guarantees can be **verified** instead of believed.

Every claim in this book carries one of three marks:

- **[code]** — it is written in the repository and the file is cited.
- **[test]** — an automated test verifies it (`server/test/e2e.ts`).
- **[pending]** — it is the production design and is not implemented yet.

Anything without a mark is context.

Contents

1. Executive summary
2. Trust architecture: where every piece of data lives
3. Protocols: why they are secure
4. The Canvas token: life cycle
5. Data minimisation and retention
6. Threat model and controls
7. Compliance: LFPDPPP and institutional policies
8. Frequently asked questions for the university
9. Pre-pilot checklist

The five guarantees, on one page

Guarantee	How it is met	Mark
FARO only reads. It cannot modify grades, submissions, due dates or content.	Every call to Canvas is a <code>GET</code> . The backend refuses any other method before touching the network. The LTI write scopes (<code>score</code> , <code>lineitem</code>) are explicitly refused.	[code] [test]
The credentials are never in the browser.	The Canvas token and the Gemini key live in <code>server/.env</code> and in the backend's memory. The extension has no field to hold them and no network permission for <code>instructure.com</code> .	[code] [test]
Only six Canvas paths, and nothing else.	An allow list in the backend (<code>server/src/allowlist.ts</code>) mirrored in the extension. A path outside the list is refused locally, with no call to Canvas.	[code] [test]
The AI mentor sees eleven fields and no personal data.	Without <code>GEMINI_API_KEY</code> on the backend (or with <code>VITE_MENTOR_MODE=local</code>), the mentor is local. If Gemini answers, it receives, through the backend, the eleven-field context, the message and the last 8 turns. The context is built in a single file (<code>src/lib/mentorContext.ts</code>) that has no access to name, email, photo or grades, and the backend filters it again (<code>server/src/mentor.ts</code>).	[code] [test]
A risk score is never shown to the student.	Friction states exist only to choose the intervention; the interface uses supportive language.	[code]

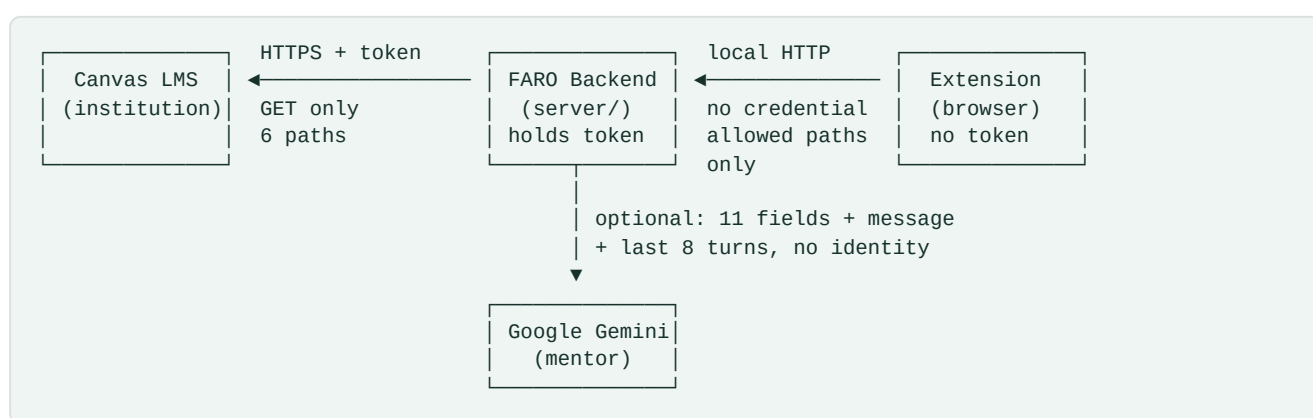
1. Executive summary

What FARO is, in terms of data

FARO is a persistence companion for adult learners studying online. It reads the course structure and the student's progress from Canvas, detects when someone is drifting away, and offers them a route back. It does not teach the course, it does not grade, it does not replace Canvas.

From the institution's point of view, FARO is **a read-only reader** of a small, well-defined slice of Canvas, with an optional artificial-intelligence layer (Google Gemini, through the backend) that receives a minimal context with no identity.

The three pieces and who talks to whom



- **Canvas** remains the academic source of truth. FARO does not write to it.
- **The backend** is the only process that knows the Canvas credential and, when the AI mentor is on, the Gemini API key. It is a Node.js server with no third-party dependencies (`server/`), which reduces supply-chain risk in the process that holds the secrets to zero.
- **The extension** is interface only. It has no token, no network permission towards Canvas, and can only ask the backend for paths the backend has already decided to allow.
- **The mentor** answers with Gemini only if the backend has `GEMINI_API_KEY` ; otherwise the local mentor answers, on the device, with no AI model. In the default mode (`VITE_MENTOR_MODE=auto`), the extension asks the backend (`GET /health` , with no student data) whether it has a mentor configured. An institution that does not approve the AI mentor builds with `VITE_MENTOR_MODE=local` or gives the server no key. When Gemini answers, open conversation goes **through the backend, never directly from the browser**, with an eleven-field context (none of which identifies the person), the student's message and the last 8 turns of the conversation. If Gemini does not answer, the local mentor does.

The security posture, in one sentence

Reduce what FARO *can* do until it matches what FARO *needs* to do, and make that reduction verifiable by an administrator without trusting us.

Current state, honestly

Component	State
Backend with token, allow list, restricted CORS, rate limit, no token leakage in logs	Implemented and tested (38 automated checks in total)
Extension with no credentials, <code>http</code> mode against the backend	Implemented
Mentor data contract (11 fields), re-checked on the backend	Implemented
Mentor with Gemini (<code>POST /api/mentor</code>): key only on the server, conversation kept out of logs, fallback to the local mentor	Implemented and tested against a fake Gemini ; not exercised against Google's real API from the development environment; the team validates it with its own key
Tests against a real Canvas (free Instructure account)	Ready to run ; needs a pilot token
Signed LTI 1.3 launch (OIDC + JWT)	Designed and documented , routes present and answering 501; needs the institution to register the tool
Per-student token via OAuth2 (instead of one shared token)	Designed ; depends on the LTI registration
PostgreSQL database with encryption at rest	Pending ; today FARO's state lives in the student's browser
Privacy review with the institution	Pending ; this book is the input

2. Trust architecture: where every piece of data lives

The question that matters most in a security review is not "which protocol do you use?" but "where is each piece of data and who can reach it?". This is the complete answer.

2.1 Trust zones

FARO has four zones. Each piece of data lives in exactly one, and crosses a boundary only for a documented reason.

Zone	What it holds	Who has access	Trust
Z1 · Canvas	The whole academic record	The institution	Highest. FARO neither controls nor alters it.
Z2 · FARO backend	The Canvas token and, when the AI mentor is on, the Gemini key (both in memory), the configuration, a short-lived cache of the launch	The backend operator	High. It is the only process with credentials. It stores no conversations.
Z3 · The student's browser	Purpose, preferences, sessions recorded by FARO, optional profile, a snapshot of the course while the tab is open	The student	Medium. It is their own device.
Z4 · AI service (Google Gemini)	For each open message to the mentor: eleven context fields, the message and the last 8 turns of the conversation	Google, under the Gemini API terms (chapter 7)	Live only when the backend has <code>GEMINI_API_KEY</code> and the extension was not built with <code>VITE_MENTOR_MODE=local</code> . Reached only through the backend. Never receives identity.

2.2 Data inventory

Each row answers: what it is, where it comes from, where it is stored, who sees it, how long it lasts.

Data that comes from Canvas (Z1 → Z2 → Z3)

Data	Canvas path	Stored in	Retention
Course name, code and end date	<code>GET /api/v1/courses/:id</code>	Browser memory while FARO is open	Discarded on close

Data	Canvas path	Stored in	Retention
Modules, their items and the student's completion state	<code>GET /api/v1/courses/:id/modules?include[]=items</code>	Same	Same
Assignments with the student's submission (date, state, score)	<code>GET /api/v1/courses/:id/assignments?include[]=submission</code>	Same	Same
Page views per hour and participations	<code>GET /api/v1/courses/:id/analytics/users/:uid/activity</code>	Same. Only the timestamps are kept ; the view counts are discarded in the mapping.	Same
The student's active courses	<code>GET /api/v1/courses?enrollment_state=active</code>	Backend, only to pick the course when none is configured	One process
Numeric id of the token owner	<code>GET /api/v1/users/self</code>	Backend, cached. Name and avatar in that response are discarded.	One process

[code] `src/data/canvas/endpoints.ts`, `src/data/canvas/client.ts` (`mapActivity`), `server/src/routes/launch.ts`.

Data FARO creates (Z3)

Data	Why it exists	Stored in	Leaves the browser
Purpose (goal + destination sentence + minutes/day)	Personalise the route and the mentor	<code>chrome.storage.local</code>	Goal, sentence and minutes are part of the mentor context. Nothing else.
Study sessions recorded by FARO	Momentum, achievements, points	<code>chrome.storage.local</code>	No

Data	Why it exists	Stored in	Leaves the browser
Finished review-card sets (date, question ids, first-try count)	Points and one offer per day	<code>chrome.storage.local</code>	No. Individual answers are not stored.
Reward redemption (tier, date)	One redemption per semester	<code>chrome.storage.local</code>	Not in the prototype (simulated). In production, to the institution so it can deliver the reward.
"Life happened" state	Choose the intervention	Memory	No
Plan accepted in the mentor (days, minutes, step ids and titles)	Show it on Home	<code>chrome.storage.local</code> (key <code>weekPlan</code>)	No. Computed locally; never goes through the model.
Mentor style, language, theme	Preferences	<code>chrome.storage.local</code>	Style and language are part of the mentor context
Name, photo (downsized), bio	Optional profile in Community	<code>chrome.storage.local</code>	Not to the mentor. In production, to other students in the course if the student enables it.
Conversation with the mentor	The current session	The extension's memory only; the backend does not store it	With the local mentor, no. When Gemini answers, each open message goes to Gemini through the backend, with the eleven-field context and the last 8 turns of the current conversation. The check-in, the plan, points and lessons without Gemini are resolved locally and are not sent to the model when they happen; their lines stay in the conversation, so they can be among the 8 turns that accompany a later open message.

[code] `src/state/storage.ts` , `src/state/store.tsx` , `src/types/index.ts` (comments on `Profile`).

Data that is never collected

This list matters as much as the ones above. FARO does **not** ask for, read or store:

- Canvas or institutional passwords.
- The student's email address.
- A list of classmates by name (the enrollments endpoint is not called; **[test]** the `/enrollments` path is refused).
- Submission contents (texts, files, exam answers).
- Other students' grades.

- Location, the student's IP address (the backend keeps only the client address for rate limiting, in memory, never persisted).
- Browsing history beyond the listed Canvas paths.

2.3 The boundaries and what crosses them

Z1 Canvas → Z2 Backend	backend's token; full response of the 6 paths
Z2 Backend → Z3 Browser	Canvas JSON, already paginated; NEVER the token; NEVER Canvas headers
Z3 Browser → Z2 Backend	the requested path (no credential); the backend checks it against the a
Z3 Browser → Z2 → Z4 AI	[when enabled] 11 fields + the message + last 8 turns; NEVER name, email
Z2 Backend → Z4 AI	the Gemini key, only in the x-goog-api-key header, never in the URL

What is not in the diagram does not happen either: the extension does not talk to Canvas (no `host_permissions` for `instructure.com` **[code]** `public/manifest.json`), and the extension does not talk to Google either: when Gemini answers, `HttpMentorService` calls `POST /api/mentor` on the backend, and it is the backend that calls Gemini with the key only it holds **[code]** `src/services/mentor.ts`, `server/src/mentor.ts`. The backend rebuilds the request from scratch (`sanitizeMentorRequest`): only the eleven fields, with bounded values; any extra field is dropped **[test]** a name, an email, an id and a grade added to the context never reach Gemini. In the default mode (`VITE_MENTOR_MODE=auto`), the extension first asks the backend `GET /health` (2 s timeout, result cached for 30 s, no student data at all) and only uses Gemini if the server reports a configured mentor **[code]** `probeGemini`, `AutoMentorService`. With `VITE_MENTOR_MODE=local`, or if the server has no `GEMINI_API_KEY` or does not answer, **no data leaves towards an AI service**: `LocalMentorService` answers on the device, with no API key. In `local` mode not even that `/health` check is made for the mentor.

2.4 Why the backend has no dependencies

The process that holds the institutional credential (and the Gemini key) uses only Node.js built-in modules (`node:http`, `node:crypto`, `node:fs`). There is no `node_modules` in production. This is not aesthetic minimalism: every third-party package running inside the token process is code the institution did not write and that could, in a compromised update, read the token from memory. The Gemini call is also a native `fetch`, with no Google SDK. Zero packages is zero surface of that kind.

[code] `server/package.json` (`"dependencies": {}`).

3. Protocols: why they are secure

FARO invents no security mechanism. It uses the same standards Canvas requires of any integration and applies them in the most restrictive way the product allows. This chapter explains each one: the problem it solves, how FARO uses it, and what it guarantees.

3.1 TLS (HTTPS) — the channel

Problem it solves: nobody on the network (a café Wi-Fi, an ISP, a badly configured institutional proxy) can read or alter what travels between FARO and Canvas.

How FARO uses it: the backend refuses to start if `CANVAS_API_URL` does not begin with `https://`. The only exception is `localhost`, and it exists solely for the automated tests against the fake Canvas.

[code] `server/src/env.ts`, `loadEnv`.

Guarantee: the Canvas token only travels encrypted. Canvas, for its part, only accepts the API over HTTPS.

3.2 Bearer token — the API credential

Problem it solves: proving to Canvas who is making the request without sending a password.

How FARO uses it: the backend adds `Authorization: Bearer <token>` to every request to Canvas, which is the method Instructure recommends ("If possible, using the HTTP Authorization header is recommended"). The token never goes in the URL, never in the body, never in a log.

[code] `server/src/canvas.ts`, `canvasGet`. **[test]** "the token was sent to Canvas on every call, and only there" and "log lines carry no query strings".

Guarantee: Instructure describes tokens as "password equivalent". FARO treats them exactly that way: they live in a single process, in memory, and are read from a file that git ignores. Chapter 4 details their life cycle.

Pilot limitation, stated: a manually generated personal token is what Instructure indicates for testing ("For testing your application before you've implemented OAuth, the simplest option is to generate an access token on your user's profile page"). For multiple users, their documentation is blunt: "Applications in use by multiple users MUST use OAuth to obtain tokens". FARO meets that with the LTI + OAuth2 path in section 3.4. **[pending]**

3.3 Path allow list — the scope

Problem it solves: a token with more permissions than needed (a student's personal token can read everything the student sees in Canvas) is used only for what FARO needs.

How FARO uses it: the backend compares every requested path against six regular expressions. `GET` only. Only the parameters `include[]`, `per_page`, `enrollment_state` and `page`, with validated

values. Anything else is refused with `403` **before touching the network**. The path is decoded before comparison, so `%2e%2e` cannot escape the pattern.

[code] `server/src/allowlist.ts`. **[test]** "a path outside the allow list is refused locally (no Canvas call)", "an unknown query parameter is refused", "path traversal is refused", "a write is refused even on an allowed path".

Guarantee: even if the extension were modified by someone, it cannot turn the backend into a general proxy to Canvas. And even though the pilot token could read the list of classmates, FARO **cannot ask for it**.

3.4 LTI 1.3 + OpenID Connect + JWT — the institutional launch

This is the production path. It is designed and documented in the code; the routes exist and answer `501` until the institution registers the tool. **[pending]**

Problem it solves: knowing with certainty *which course* and *which person* are opening FARO, without asking anyone to identify themselves and without FARO having to trust what the extension says.

How it works (IMS Security Framework 1.0):

1. Canvas initiates an OpenID Connect *login* towards `GET /lti/login`. FARO stores a short-lived `state` and `nonce` and redirects to Canvas's authorisation endpoint.
2. Canvas answers with `POST /lti/launch` and an `id_token`: a JWT **signed by Canvas** with RS256.
3. FARO downloads Canvas's public keys (JWKS), verifies the signature, the issuer (`iss`), the audience (`aud` = FARO's `client_id`), the expiry, and that the `nonce` is the one issued and has not been used.
4. From the verified claims, FARO reads exactly two things: the course (`https://purl.imsglobal.org/spec/lti/claim/context`) and the person (`sub`).
5. FARO issues its own session (an `HttpOnly`, `Secure`, `SameSite=None` cookie, because it runs inside a Canvas iframe) bound to that course and that person.

[code] `server/src/routes/lti.ts` (full specification in the comments).

Guarantee: the student's identity is asserted cryptographically by Canvas, not by the extension. FARO cannot be tricked into showing someone else's course because it accepts no identity claim that is not signed by the institution.

LTI scopes: FARO requests `contextmembership.readonly` (NRPS, for aggregate presence counts) and `result.readonly` (AGS, read-only results). It **refuses** `score` and `lineitem`, the two scopes that would allow writing to the gradebook. An administrator can verify this on the developer key screen without reading a line of code.

[code] `src/data/canvas/endpoints.ts`, `ltiScopes`.

3.5 OAuth2 with a developer key — the per-student token

[pending] Once the tool is registered, each student obtains their own access token through Canvas's OAuth2 flow, with scopes limited to the six paths (in the `url:GET|/api/v1/...` form Canvas uses on the developer key). Tokens from this flow expire in one hour and are renewed with a *refresh token*, both stored only in the backend. They are revoked with `DELETE /login/oauth2/token`.

Guarantee: the pilot's shared token disappears. Each student sees exactly what Canvas lets them see, and revoking FARO's access for one person is a single-token operation.

[code] the scopes are already written in `src/data/canvas/endpoints.ts`, `requiredScopes`.

3.6 CORS — who may call the backend from a browser

Problem it solves: any web page open in the same browser using the FARO backend as if it were the extension.

How FARO uses it: the backend keeps a list of allowed origins (`FARO_ALLOWED_ORIGINS`). An origin outside the list receives `403` with no data and no CORS headers. The advertised methods are `GET`, `POST`, `OPTIONS`: `POST` exists for `POST /api/mentor` (and for the LTI launch, which answers 501 today). Towards Canvas the backend still sends only `GET`.

Exception for local development: when the backend listens on loopback (`FARO_HOST = 127.0.0.1`, `localhost` or `::1`, the default), it also accepts any Chrome extension origin of the form `chrome-extension://<32 letters a-p>`, so the unpacked extension works without copying its id into `FARO_ALLOWED_ORIGINS`. It is turned off with `FARO_STRICT_ORIGINS=true`, and turns itself off when the server listens on another interface (e.g. `0.0.0.0`). At start-up the backend says so: *"Accepting the unpacked FARO extension (any chrome-extension:// origin, loopback only)"*. **In production:** list the exact extension id and set `FARO_STRICT_ORIGINS=true`.

[code] `server/src/http.ts`, `corsHeaders`; `server/src/env.ts` (`allowExtensionOrigins`). **[test]** "a browser origin outside the list gets 403 and no data", "CORS preflight succeeds for an allowed origin", "an extension origin is refused unless extension origins are enabled", "mentor: the unpacked extension can call it when extension origins are on (loopback dev)".

3.7 Rate limit — protection against loops

Problem it solves: an extension with a bug that retries in a loop could send thousands of requests to Canvas with the institutional token, and Canvas could suspend that token.

How FARO uses it: a per-client counter over a one-minute window (`FARO_RATE_LIMIT_PER_MINUTE`, 120 by default). Beyond it, `429` with `Retry-After`. Model calls also have their own per-client cap (`FARO_MENTOR_PER_MINUTE`, 20 by default), which answers `429 mentor_rate_limited` before calling Gemini, to contain cost.

[code] `server/src/http.ts`, `RateLimiter`; `server/src/routes/mentor.ts`. **[test]** "rate limit trips after the configured burst", "mentor: per-client cap answers 429 before calling the model".

3.8 Response headers

Every backend response carries `Cache-Control: no-store` (no academic data stays in intermediate caches), `X-Content-Type-Options: nosniff` and `Referrer-Policy: no-referrer`.

[code] `server/src/http.ts`, `send`. **[test]** "responses carry no-store and nosniff".

3.9 Summary: which protocol protects what

Threat	Protocol / control	State
Eavesdropping on the network	Mandatory TLS	[code]
Credential theft from the browser	The browser has no credential	[code] [test]
Token used beyond what is needed	Allow list of 6 paths, GET only	[code] [test]
Student impersonation	LTI 1.3 with a JWT signed by Canvas	[pending]
Token shared between people	OAuth2 per student	[pending]
Malicious page using the backend	CORS with an origin list	[code] [test]
Request loop	Rate limit	[code] [test]
Writing to the academic record	No write scopes; <code>score / lineitem</code> refused	[code]

4. The Canvas token: life cycle

The token is the most sensitive piece of data FARO handles. This chapter follows its whole life: where it is born, where it lives, where it travels, where it is **never**, and how it dies. Section 4.8 applies the same rules to the backend's second secret: the Gemini API key.

4.1 Birth

Pilot (today): generated by the owner of the Canvas account, under *Account* → *Settings* → *Approved Integrations* → *New Access Token*. Canvas shows it only once. Two Canvas details work in our favour:

- Institutions can **require an expiry date** for student tokens (Instructure documents a default of 30 days for students and 90 for instructors), and can disable manual token generation altogether.
- The token inherits exactly the permissions of the user who created it: a student's token cannot see more than that student.

Production: issued by Canvas through OAuth2 from a developer key registered by the institution, with limited scopes and a one-hour expiry. **[pending]**

4.2 Residence

The token lives in **two places and only two**:

1. `server/.env`, a file on the machine that runs the backend. It is in `.gitignore` **[code]** and the repository ships `server/.env.example` with the field empty so nobody has to guess the format.
2. The backend process's memory, in the `Env` object, from start-up to shutdown.

4.3 Travel

It leaves the backend in a single direction: towards Canvas, in the `Authorization` header, over HTTPS, and only for allow-listed paths. Before following a pagination link (`Link: rel="next"`), the backend verifies the URL still points at the configured Canvas host; a link pointing to another domain is refused, because following it would send the token elsewhere.

[code] `server/src/canvas.ts`, the `url.startsWith(env.canvasApiUrl + '/')` check.

4.4 Where it never is

Place	Why not	Verification
The extension	<code>CanvasConfig</code> has no token field. The variable does not exist.	<code>src/data/canvas/config.ts</code>
The Vite build	Vite only exposes <code>VITE_*</code> variables; the token has no such prefix and is not in the root <code>.env</code> .	The root <code>.env.example</code> says so explicitly

Place	Why not	Verification
Backend logs	The logger receives method, path without query, status, duration and an id. Never headers.	[test] "the token was sent to Canvas on every call, and only there"
Backend responses	<code>describe(env)</code> is the only representation of the configuration that leaves, and it contains <code>tokenPresent: true/false</code> , not the token.	[test] "health names the Canvas host and never the token"
Error messages	<code>CanvasError</code> stores status and path, never the body of Canvas's response.	<code>server/src/canvas.ts</code>
The git repository	<code>.gitignore</code> excludes <code>.env</code> in any folder.	<code>.gitignore</code>
The AI service	The mentor context has no credential field, and the Gemini call carries only the Gemini key.	<code>src/lib/mentorContext.ts</code> , <code>server/src/mentor.ts</code>

4.5 Rotation

Changing the token is editing `server/.env` and restarting the backend. There is no token cache anywhere else that survives a restart. Pilot recommendation: tokens with a 30-day expiry, rotated when they expire.

4.6 Revocation

- **From Canvas:** the token owner deletes it under *Approved Integrations*. From that instant the backend receives `401` and reports it as `canvas_token_rejected` without trying anything else. **[test]** "a wrong token is reported as rejected, not as a generic failure".
- **From the backend:** clear the value in `.env` and restart. The backend refuses to start with half a Canvas configuration (URL without token or token without URL) and refuses to start when it has nothing to serve (neither Canvas nor mentor). If only the Gemini key is left, it starts mentor-only and the Canvas routes answer `503 canvas_not_configured`; `/health` shows it as `canvas: null`. So a "started without a credential and nobody noticed" state does not exist. **[test]** "config: half a Canvas setup is refused, mentor-only is accepted".
- **Production (OAuth2):** `DELETE /login/oauth2/token` revokes the student's token in Canvas. **[pending]**

4.7 If the token leaked

Scenario: someone gains access to `server/.env`.

1. **Blast radius:** reading what that Canvas user can read. No writing: a student's token cannot alter their record, and an instructor's cannot either through FARO, although it could directly against Canvas. That is why the pilot uses **student-account tokens**, never instructor or administrator ones.
2. **Containment:** revoke the token in Canvas (one click). Generate a new one. Restart the backend.

3. **Detection:** Canvas records the use of every token; a usage pattern from an unknown IP is visible to the institution.
4. **Structural prevention:** the LTI + OAuth2 path removes the long-lived `.env` token file and replaces it with one-hour tokens issued per student.

4.8 The second secret: the Gemini key

If the institution turns on the AI mentor, the backend holds a second secret: `GEMINI_API_KEY`. It follows exactly the Canvas token's rules.

- **Birth:** created in Google AI Studio, in a Google Cloud project **with active billing** (chapter 7 explains why the free tier is not usable with real students).
- **Residence:** `server/.env` and the backend process's memory. Never in the extension: the root `.env.example` says so explicitly, and `HttpMentorService` has nowhere to hold one. It is optional: empty, the AI mentor is off and the extension uses its local mentor.
- **Travel:** it leaves only towards Google, in the `x-goog-api-key` header of the call to `models/{model}:generateContent`, **never in the URL** (URLs end up in proxy logs). **[test]** "mentor: the key goes in a header, never in the URL".
- **Where it never is:** in the backend's logs or responses. `describe(env)` shows only `mentor: {provider, model}`. If Google rejects the key (401/403), the backend answers `502 mentor_key_rejected` without repeating Google's error body. **[test]** "mentor: a rejected key and a safety block map to fallback codes", "mentor: the key and the conversation never appear in the log".
- **Rotation and revocation:** edit `server/.env` and restart. If the key is exposed, revoke it in Google AI Studio / Google Cloud and generate another.
- **Blast radius if it leaks:** someone could make Gemini calls billed to the institution's project. It gives no access to Canvas or to student data: FARO stores no conversations. `FARO_MENTOR_PER_MINUTE` (20 by default) caps, per client, how many model calls the backend accepts, to contain cost.

5. Data minimisation and retention

FARO's guiding principle for data is simple: **collect a piece of data not because it is technically possible, but because a specific feature needs it**. This chapter shows how that applies in each layer, and what happens to data over time.

5.1 Minimisation in the Canvas layer

Canvas returns far more than FARO uses. The integration layer discards what it does not need **at the moment of reading it**, not afterwards.

What Canvas sends	What FARO keeps	What is discarded
Course object (dozens of fields)	id, name, code, start and end dates	Everything else: settings, permissions, image, etc.
Modules with items	id, name, position, state, item count, each item's <code>content_id</code> and whether it is completed	Page titles, external URLs, files
Assignments with submission	id, name, due date, points possible, and from the submission: date, state, score	Assignment description, rubric, attachments, instructor comments
Student activity	Timestamps only. The number of views per hour is removed.	View counts, visited URLs
<code>users/self</code>	The numeric id	Name, avatar, email, time zone

[code] `src/data/canvas/raw.ts` declares only the fields that are read; `src/data/canvas/client.ts` does the mapping; `server/src/routes/launch.ts` keeps only `self.data.id`.

Why the activity case matters: "how many pages they viewed at 3 pm" is surveillance data. "When they last came in" is the only thing the friction engine needs. FARO keeps the second and destroys the first.

5.2 Minimisation in the AI mentor

The context the model receives is built in a single function and contains exactly these eleven fields:

```
{
  "course": "Gestión de Proyectos",
  "progress": 44,
  "next_activity": "Caso de estudio: alcance",
  "estimated_time": 25,
  "student_goal": "career_growth",
  "destination": "Liderar mis propios proyectos",
  "available_time": 20,
  "momentum": 62,
  "friction_state": "FRICTION",
  "style": "encouraging",
  "language": "es"
}
```


No name, email, Canvas id, grades, photo, history of other conversations or dropout-risk classification. The friction state is one of five intervention labels (`FLOWING` , `FRICTION` , `POSSIBLE_OVERWHELM` , `DISCONNECTION` , `RECOVERY`), not a score.

The only human-authored information is the **destination sentence**, which the student writes freely at onboarding. The example shown is *"Lead my own projects instead of just executing them"*, steering towards a goal, not personal data. In production this sentence must carry an explicit warning: *"do not include anything that identifies you"*. **[pending]**

[code] `src/lib/mentorContext.ts` , `src/types/index.ts` (`MentorContext` interface).

When the mentor uses Gemini (the backend has `GEMINI_API_KEY` and the extension was not built with `VITE_MENTOR_MODE=local`), what leaves for Google, always through the backend and never directly from the browser, is: these eleven fields, the student's message and the last 8 turns of the current conversation (role and text, nothing else). The backend does not trust the extension to have done its part: `sanitizeMentorRequest` rebuilds the request with only the eleven fields, checks the enumerated values, caps lengths and drops any extra field **[test]** "mentor: extra fields in the context never reach Gemini", "mentor: history is trimmed to the last 8 turns".

The structured parts never go through the model: the check-in ("I noticed something changed..."), Focus sessions, the weekend plan and answers about points are computed in the extension from real data and are not sent to the model. Their lines do stay in the conversation, so they can travel as part of the last 8 turns if the student then writes an open message. A "do my graded work" request is answered by the backend itself without calling the model **[test]** "mentor: "do my homework" is refused before any model call".

[code] `server/src/mentor.ts` .

5.3 Minimisation in Community

Community is the part of FARO with the greatest temptation to collect social data, which is why it has the strictest rules, written into the types:

- **There is no follower field and no rank.** They are not hidden: they are not in the data model, so no function can compute them.
- **Presence is aggregate.** "23 students studying now" is a count. "Andrea has been at it for 37 minutes" never appears, and the `CoursePresence` type only admits numbers.
- **Classmates have no photo.** Only initials and a decorative colour. Only the student themselves can add an image to their avatar, downsized in their browser before being saved.
- **Reactions are shown separately**, never summed into a score.

[code] `src/types/index.ts` , comments on `CommunityAuthor` and `CoursePresence` .

5.4 Retention

Data	Where	How long	How it is deleted
Course snapshot (from Canvas)	Browser memory	While FARO is open	Closing the tab or panel
Canvas request log	Browser memory, 30 entries	While FARO is open	Automatic; never persisted or sent
Purpose, preferences, sessions, profile, plan accepted in the mentor (<code>weekPlan</code>)	<code>chrome.storage.local</code>	Until the student deletes it	<i>Reset</i> button in Progress, or uninstalling the extension; the plan also with <i>Clear plan</i> on Home
Conversation with the mentor	The extension's memory	The session	Closing FARO or <i>Reset</i> . The backend neither stores nor logs it
What is sent to Gemini (when Gemini answers)	Google	Under the Gemini API terms (chapter 7)	With a paid key, Google logs it for a limited time only for abuse detection and legal requirements
Canvas token	<code>server/.env</code> + backend memory	Until rotation or revocation	Edit <code>.env</code> and restart; revoke in Canvas
Gemini key	<code>server/.env</code> + backend memory	Until rotation or revocation	Edit <code>.env</code> and restart; revoke in Google
Launch cache (course id, user id)	Backend memory	Until the process restarts	Restart
Backend logs	Standard output	Defined by the operator	Defined by the operator; they contain no personal data, query strings, request bodies or anything from the mentor conversation

Production [pending]: once PostgreSQL exists, the proposal is:

- Study sessions, purpose and preferences: while the enrolment is active, plus 90 days to allow "returning" to a paused course; then automatic deletion.
- Mentor conversations: not stored beyond the session unless explicit consent is given to improve the product, in which case they are anonymised.
- Friction events and interventions: aggregated for pilot metrics, with the student id replaced by a pseudonymous id that only the institution can resolve.

5.5 What the student can see and control

- **Profile** → **Canvas** shows, live, every call FARO makes, with its path, status and latency. Transparency is not a document: it is a screen.
- **Profile** → **Canvas** shows the LTI scopes FARO refuses.

- **FARO Mentor** says, at the bottom, what the mentor sees and who answers: *"Answers by Gemini, through FARO's server..."* or *"Local mentor: no AI model, nothing leaves your device."* Messages written by Gemini carry a violet ring on FARO's avatar.
- **Progress** → **Reset** wipes all of FARO's local state.
- Nothing in the profile is mandatory. FARO works fully without a name or photo.

6. Threat model and controls

A threat model answers three questions: what we want to protect, from whom, and what we have done about it. This table is the answer, threat by threat. The *Test* column names the automated check in `server/test/e2e.ts` when one exists.

6.1 Assets

1. **The Canvas token** — grants read access to its owner's record.
2. **The student's academic data** — progress, submissions, dates.
3. **The student's identity** — who the person behind a FARO session is.
4. **The integrity of the record** — nothing FARO does can alter Canvas.
5. **The student's trust** — that FARO does not turn into surveillance.
6. **The Gemini key** — when the AI mentor is on, it allows model calls billed to the institution's project.

6.2 Threats and controls

#	Threat	Asset	Control	State	Test
T1	An attacker extracts the credential from the extension's code (extensions are public code)	Token	There is no credential in the extension. <code>CanvasConfig</code> has no field for it.	[code]	—
T2	A modified extension uses the backend to read any Canvas path	Data	Allow list of 6 paths in the backend; local <code>403</code> with no Canvas call	[code]	"a path outside the allow list is refused locally"
T3	A modified extension tries to write to Canvas through the backend	Integrity	<code>GET</code> only; <code>405</code> for other methods; no write scopes	[code]	"a write is refused even on an allowed path"
T4	Parameter tampering to widen Canvas's response (<code>all_dates</code> , <code>include[]=user</code>)	Data	Parameters and values validated; the rest refused	[code]	"an unknown query parameter is refused"
T5	Encoded <i>path traversal</i> (<code>..%2F</code>) to escape the pattern	Data	Decoding before comparison; <code>..</code> and <code>//</code> refused	[code]	"path traversal is refused"

#	Threat	Asset	Control	State	Test
T6	A malicious web page in the same browser calls the backend	Data	CORS with an origin list; unknown origin → 403 with no data	[code]	"a browser origin outside the list gets 403"
T7	A request loop exhausts the Canvas quota and exposes the token to suspension	Token	Per-client rate limit, 429	[code]	"rate limit trips after the configured burst"
T8	The token appears in logs, errors or responses	Token	Logger without headers or query; errors without the Canvas body; describe() without the token	[code]	"health names the Canvas host and never the token", "log lines carry no query strings"
T9	A tampered pagination Link redirects the token to another domain	Token	The backend verifies each page stays on the configured Canvas host	[code]	—
T10	Network eavesdropping between backend and Canvas	Token, data	Mandatory HTTPS; the backend does not start with http:// outside localhost	[code]	—
T11	Impersonation: someone claims to be another student	Identity	Pilot: the backend infers identity from the token, not from the extension. Production: JWT signed by Canvas (LTI 1.3)	[code] / [pending]	"launch resolves the token owner"
T12	Intermediate caches keep academic data	Data	Cache-Control: no-store on every response	[code]	"responses carry no-store and nosniff"
T13	A compromised third-party package reads the token from the backend's memory	Token	Zero dependencies in the backend	[code]	—
T14	The AI service receives personal data	Identity, trust	Eleven-field context built in a single function; no name, email, ids, grades. The backend rebuilds it (sanitizeMentorRequest) and drops any extra field; history capped at 8 turns	[code]	"mentor: extra fields in the context never reach Gemini", "mentor: history is trimmed to the last 8 turns"
T15	FARO becomes a surveillance tool (who studies how much)	Trust	Aggregate presence in the data model; no ranking or followers; view counts discarded	[code]	—

#	Threat	Asset	Control	State	Test
T16	A dropout-risk classification is exposed to the student	Trust	Friction states are internal; the interface uses supportive language; no risk score exists	[code]	—
T17	An instructor or admin token in the pilot widens the blast radius of a leak	Token, data	Policy: the pilot uses student-account tokens only	Policy	—
T18	The backend starts without a credential and serves empty data as if real	Trust	Refuses to start with half a Canvas configuration or with nothing to serve; in mentor-only mode the Canvas routes answer 503 canvas_not_configured ; with an invalid token answers canvas_token_rejected	[code]	"a wrong token is reported as rejected", "mentor-only server: Canvas routes say canvas_not_configured", "config: half a Canvas setup is refused, mentor-only is accepted"
T19	A student account cannot read analytics and FARO invents activity	Trust	401 / 403 is passed to the client; FARO uses submission dates and says so on screen	[code]	"Canvas 401 on analytics is passed through"
T20	Theft of the server/.env file	Token, Gemini key	Scope limited to reading one student; one-click revocation in Canvas; key revocation in Google; the OAuth2 path removes the token from the file	Policy / [pending]	—
T21	The Gemini key leaks through the URL, logs, a response or an error	Gemini key	Only in the x-goog-api-key header; never in the URL; describe() shows only provider and model; Google's error body is never echoed or logged	[code]	"mentor: the key goes in a header, never in the URL", "mentor: a rejected key and a safety block map to fallback codes", "mentor: the key and the conversation never appear in the log"
T22	Mentor conversations end up in the backend logs	Identity, trust	The logger only receives method, path, status, duration and id; never bodies. Not the message, not the reply, not the context	[code]	"mentor: the key and the conversation never appear in the log"
T23	A client triggers model calls in a loop and runs up cost	Gemini key	Per-client cap on model calls (FARO_MENTOR_PER_MINUTE , 20) on top of the global limit; 32 KB maximum body	[code]	"mentor: per-client cap answers 429 before calling the model", "mentor: a non-JSON body gets 415 and a huge body 413"

#	Threat	Asset	Control	State	Test
T24	The mentor does the student's graded work	Trust, academic integrity	"Do my homework" requests are answered by the backend without calling the model; the system prompt forbids producing graded work	[code]	"mentor: "do my homework" is refused before any model call"
T25	Google uses the conversations to improve its products, or human reviewers read them	Identity, trust	Policy: only keys from a project with active billing (paid services); never the free tier with real students (chapter 7)	Policy	—
T26	Another extension installed in the same browser calls the local backend (on loopback any chrome-extension:// origin is accepted)	Data, Gemini key	Only when the server listens on loopback; off with <code>FARO_STRICT_ORIGINS=true</code> and by itself on any other interface; the mentor's per-client cap still applies; in deployment, exact extension id + <code>FARO_STRICT_ORIGINS=true</code>	[code] / Policy	"an extension origin is refused unless extension origins are enabled", "config: half a Canvas setup is refused, mentor-only is accepted"

6.3 What this model does not cover yet

- **Authentication of the extension to the backend in production.** Today the backend trusts the origin (CORS) and listens only on `127.0.0.1`. In an institutional deployment, the session issued after the LTI launch is what authorises each call. **[pending]**
- **Encryption at rest** of the future database. **[pending]**
- **Access auditing** by the institution: who queried what and when. The backend logs path and status per request; persistence and a pseudonymous id are missing. **[pending]**
- **External security review.** This document is a self-assessment; an institutional pilot must include a review by the university's security team.

7. Compliance: LFPDPPP and institutional policies

This chapter is technical guidance to prepare the institution's legal review. It does not replace the opinion of the university's legal department or data-protection officer.

7.1 The applicable framework in Mexico

The **Ley Federal de Protección de Datos Personales en Posesión de los Particulares (LFPDPPP)** was published in the Diario Oficial de la Federación on 20 March 2025 and entered into force on 21 March 2025. It replaces the 2010 law. The regulator for the private sector is no longer INAI, which was dissolved, but the **Secretaría Anticorrupción y Buen Gobierno**.

Three points of the new law that touch FARO directly:

- 1. **Definition of personal data.** Any information about an identified *or identifiable* person, "when their identity can be determined directly or indirectly through any information". A numeric Canvas id is therefore personal data even without a name: it is indirectly identifiable by the institution.
- 2. **Privacy notice.** It must identify sensitive data and distinguish the purposes that require express consent. When data is collected electronically, a simplified notice must be offered that refers to the full one.
- 3. **Consent.** It may be express or tacit; it is tacit when, the privacy notice having been made available, the data subject does not object. Sensitive data still requires express consent.

Note on the regulated party. A private university such as Tecmilenio is a private party and the LFPDPPP applies. A public university is governed by the Ley General de Protección de Datos Personales en Posesión de Sujetos Obligados. Both share the principles; this chapter's analysis holds for both, but the instrument and the authority change. **The institution must confirm which applies.**

7.2 The principles, one by one

Principle	What it requires	How FARO meets it	Mark
Lawfulness	Process data according to law and without deception	FARO only reads what the student can already see about themselves in Canvas; the request log is visible on screen	[code]
Consent	Have the data subject's will	Pilot: express, informed consent from each participant (template in 7.4). Production: privacy notice embedded in the LTI launch	Policy / [pending]

Principle	What it requires	How FARO meets it	Mark
Information	An accessible privacy notice	The <i>Profile</i> → <i>Canvas</i> screen and the Mentor footer show what is queried, what is sent and who answers (Gemini or the local mentor); the formal notice is to be drafted with the institution	[code] / [pending]
Quality	Accurate, up-to-date data	Canvas is the source of truth; FARO does not copy the record, it reads it on each open	[code]
Purpose	Use data only for the stated purpose	Every Canvas path has a purpose written next to it (<code>server/src/allowlist.ts</code> , <code>purpose</code> field); there is no secondary use	[code]
Loyalty	No data obtained by deceptive means	No tracking beyond the 6 paths; no <i>dark patterns</i> to obtain the profile (everything is optional)	[code]
Proportionality	Only the data that is necessary	Chapter 5: view counts discarded, name and avatar from <code>users/self</code> discarded, mentor with 11 fields re-checked on the backend and history capped at 8 turns	[code] [test]
Accountability	Ensure compliance and be able to demonstrate it	This book; the 38 automated tests; the checklist in chapter 9	[test]

7.3 Sensitive data

FARO **does not process sensitive data** as the law defines it (ethnic origin, health, beliefs, political opinions, sexual preference, genetic data). Two nuances the institution should consider:

- The *"life happened"* state (less time, overwhelmed, need a break) is self-declared, used only to choose an intervention, and **never persisted**. It is not one of the eleven context fields. If the student picks it in the mentor's check-in, that choice stays as a turn of the conversation, and when Gemini answers it can travel among the last 8 turns of a later open message. It is not health data, but the privacy notice should mention it to leave no doubt.
- The **destination sentence** is free text. If a student wrote sensitive data there, FARO would store it locally and include it in the mentor context (and, when Gemini answers, it would reach Google). Mitigation: an on-screen warning **[pending]** and a policy of not storing conversations.
- **Messages to the mentor** are free text too. When Gemini answers, they go to Google through the backend. If a student mentions a crisis or wanting to hurt themselves, the mentor's rules tell it to respond with care, stop the study talk and encourage them to contact the university's student support or emergency services right away. FARO is not a psychological support service; the institution must decide which channel to name.

7.4 Consent for the pilot

For a pilot with five to thirty students, written express consent is recommended. Minimum elements it must contain:

1. Who the controller is (the institution) and who the processor is (the FARO team).
2. What data is read from Canvas (the six paths, in plain language).
3. What data FARO creates (purpose, sessions, preferences, optional profile).
4. That FARO modifies nothing in Canvas.
5. If the institution enables Gemini: that the mentor may answer with a Google model; that it receives, through FARO's backend, a context with no name or email, the student's message and the last 8 turns of the conversation; that FARO does not store conversations; and that the mentor screen always says who answers. That only people aged 18 or over take part.
6. How long data is kept and how to request deletion (*Progress* → *Reset*, and token revocation).
7. That participation is voluntary and withdrawing does not affect grades.
8. How to exercise ARCO rights before the institution.

7.5 ARCO rights

Right	How it is exercised in FARO today	In production
Access	<i>Profile</i> shows everything FARO stores about the student; <i>Profile</i> → <i>Canvas</i> shows everything it reads	Export from the backend
Rectification	Academic data is corrected in Canvas (source of truth); FARO data, in <i>Profile</i>	Same
Cancellation	<i>Progress</i> → <i>Reset</i> wipes local state; uninstalling the extension removes everything	Database deletion on request
Opposition	Do not install the extension; withdraw pilot consent	Disable the LTI tool for that student

7.6 Transfers

- **To Canvas (Instructure):** FARO transfers nothing to Canvas; it only reads. The institution's relationship with Instructure is already covered by its existing contract.
- **To the AI model provider (Google, Gemini API):** only if the backend has `GEMINI_API_KEY` (in the default mode, `VITE_MENTOR_MODE=auto`, the extension detects it by itself; with `gemini`, it always tries). An institution that does not approve the AI mentor builds with `VITE_MENTOR_MODE=local` or gives the server no key. It is a transfer of the 11 context fields, the student's message and the last 8 turns of the current conversation; never name, email, Canvas ids, grades or photo. It must appear in the privacy notice. How that data may be used depends on the kind of key (section 7.8). With `VITE_MENTOR_MODE=local`, or without a key on the server, there is no transfer.
- **Between the student's devices [pending]:** once the backend with a database exists, FARO's state will sync through it. That is processing by the processor, not a transfer to a third party.

7.7 Institutional Canvas policies

Regardless of the law, the institution controls in Canvas:

- Whether students can generate access tokens and with what expiry.
- Which scopes it grants to FARO's developer key (recommendation: exactly those in `requiredScopes`).
- Whether the LTI tool is installed at account, sub-account or course level.
- Whether students can view their own analytics (this determines whether FARO uses the activity endpoint or the submission-based fallback).

None of these decisions requires changes in FARO; the backend respects what Canvas allows and reports what it does not.

7.8 The Gemini API terms

Source: *Gemini API Additional Terms of Service*, ai.google.dev/gemini-api/terms . The institution must review the version in force before the pilot; what follows is what FARO assumes.

	Paid Services (Google Cloud project with active billing)	Unpaid Services (no billing)
Does Google use prompts and responses to improve its products?	No	Yes
What does Google do with them?	Logs them for a limited time, only for abuse detection and legal requirements	Uses them to improve products; human reviewers may read them
Warning in the terms	—	Do not submit personal information

FARO's rule: with real students, only a key from a project with **active billing**. **Never** the free tier. The free tier is for the team to test with made-up data, nothing more.

Age. The terms require Gemini API users to be 18 or over. The pilot is with adult students, 18 or over; the institution must confirm it when recruiting.

What FARO adds on its own, independent of Google's terms: the key lives only on the backend; the backend neither stores nor logs conversations; the context is filtered twice (extension and backend); and if Gemini fails the local mentor answers, so the service does not depend on Google to keep working.

8. Frequently asked questions for the university

The questions an IT department, a data-protection officer or an academic director asks before connecting a tool to Canvas, with short answers and the reference to check them.

Access and scope

Can FARO modify grades, submissions or due dates? No. Every call to Canvas is a read (`GET`). The backend refuses any other method before contacting Canvas, and the LTI scopes that would allow writing to the gradebook (`score` , `lineitem`) are explicitly refused. Check `server/src/allowlist.ts` and the developer key screen in Canvas.

What exactly does FARO see in Canvas? Six paths: the course, its modules with items, the assignments with the student's own submission, the student's own activity (timestamps), the student's list of active courses, and their numeric id. Nothing else. The list is a thirty-line file:

```
server/src/allowlist.ts.
```

Does FARO see other students' data? No. It does not call the enrollments endpoint or the course users endpoint. In production, the "how many are studying" count will come from LTI's NRPS service in aggregate mode, with no names.

Can FARO see the contents of a submission or an exam? No. It reads the submission's date, state and score, not its contents. The fields that are read are declared in `src/data/canvas/raw.ts` ; whatever is not there is ignored.

Can a student use FARO to see someone else's course? No. In the pilot, the backend infers identity from the token, not from what the extension claims. In production, identity is signed by Canvas (LTI 1.3) and FARO accepts no other.

Credentials

Where is the Canvas token? In `server/.env` , on the machine running the backend, and in that process's memory. Nowhere else. The extension has no field for a token and no network permission for `instructure.com` .

Is the token in the source code or on GitHub? No. `.env` is in `.gitignore` . The repository contains `server/.env.example` with the field empty.

Where is the Gemini key? Like the token: in `server/.env` and in the backend's memory, and only if the AI mentor is wanted. Never in the extension. It travels to Google only in the `x-goog-api-key` header, never in the URL, and does not appear in logs or responses. If it is exposed, revoke it in Google and generate another. Chapter 4.8.

What if the token leaks? Revoke it in Canvas with one click, generate another, restart the backend. The blast radius is reading what that student sees about themselves. That is why the pilot uses only student-account tokens, never instructor or administrator ones. Chapter 4.

Does every student need to generate a token? In the pilot, yes, one per participant (or a test-account token). That is the practice Instructure indicates for testing. For a real deployment, FARO uses OAuth2 with a developer key and Canvas issues tokens automatically, valid for one hour.

Artificial intelligence

Which model does FARO use? Google Gemini (`GEMINI_MODEL` , by default `gemini-3.5-flash` ; model names change and must be checked at `ai.google.dev/gemini-api/docs/models`), but only if the backend has `GEMINI_API_KEY` . Otherwise none: the local mentor answers, deterministically, and nothing leaves the device. `VITE_MENTOR_MODE` has three values: `auto` (default: the extension asks the backend's `GET /health` , with a 2 s timeout and the result cached for 30 s, no student data, and uses Gemini if the server has it), `local` (never leaves the device, not even that check) and `gemini` (always tries it). The mentor screen always says which of the two answers, based on that live check.

What does the AI model receive? Only when Gemini answers, and always through the backend: eleven fields (course name, progress percentage, next activity and its duration, declared goal, destination sentence, available minutes, momentum, friction state, style and language), the student's message and the last 8 turns of the current conversation. The context is built in `src/lib/mentorContext.ts` and the backend filters it again in `server/src/mentor.ts` . It does not receive name, email, Canvas ids, grades, photo or earlier conversations.

Can the model do the student's homework? No. There is a filter for "do my exam"-type requests that answers with a refusal and offers to break the task down, explain the concept or review the reasoning; when Gemini is on, the backend answers those requests without calling the model. On top of that, the system prompt's rules forbid producing graded work. The filter is the same across the five communication styles.

Does data go to an AI provider today? Only if the backend has `GEMINI_API_KEY` and the extension was not built with `VITE_MENTOR_MODE=local` . Without a key, or with `local` , no: the mentor is local and deterministic. An institution that does not approve the AI mentor builds with `VITE_MENTOR_MODE=local` or gives the server no key. Before using it with real students, an updated privacy notice is needed (chapter 7).

Does Gemini train on student data? Not if the key belongs to a Google Cloud project with active billing (the Gemini API's Paid Services): then Google does not use prompts or responses to improve its products, and only logs them for a limited time for abuse detection and legal requirements. On the free tier it does use them to improve products, human reviewers may read them, and the terms say not to submit personal information; that is why **the free tier is never used with real students**. Source: `ai.google.dev/gemini-api/terms` . Chapter 7.8.

What if Gemini is down? The local mentor answers, and the student sees it. If the server is down or has no key, the mentor footer says *"Local mentor..."* from the moment it opens. If the server had Gemini but a reply fails, that reply carries the note *"Gemini could not answer just now, so the local mentor did"*. The backend returns clear codes for each failure (`503 mentor_not_configured` , `502 mentor_key_rejected` , `502 mentor_upstream` , `502 mentor_blocked` , `504 mentor_timeout` , `429 mentor_rate_limited`) and the extension falls back to the local mentor on any of them. The student always gets a reply; their progress does not depend on Google.

Does FARO store mentor conversations? No. They live in the extension's memory for the session. The backend neither stores nor logs them: not the message, not the reply, not the context.

Surveillance and wellbeing

Does FARO compute a student's "dropout risk"? It computes internal friction states (five labels based on days without activity and pending work) to decide which intervention to offer. It **never** shows a risk score to the student and shares it with nobody. The interface says "your route needs attention", not "risk: 87%".

Can an instructor see who studies how much? That screen does not exist, and Community's data model only admits aggregate counts, not people. If the institution wanted pilot metrics, the proposal is to deliver them aggregated and with pseudonymous ids.

Does FARO send notifications or reminders? No. There are no push notifications, emails or reminders. FARO waits for the student to open the panel. The "safe harbour" (planned pause) explicitly has no countdown.

Is FARO a video game? No. It uses mechanics (route, checkpoints, achievements) without lives, levels or leaderboards. The five achievements and five recognitions require having persisted; none is earned by clicking.

Operations

Where does the backend run? In the pilot, on a team or institution machine, listening on `127.0.0.1`. In production, on institutional infrastructure or an approved provider, behind TLS, with the LTI launch as authentication.

What dependencies does it have? The backend, none. Only Node.js. The extension, two: `react` and `react-dom`.

How do I know the guarantees in this book are true? Run `npm test` inside `server/`: thirty-eight automated checks against a fake Canvas with real pagination and a fake Gemini. And open *Profile* → *Canvas* in FARO: the live request log shows every call.

What is missing for production? LTI 1.3 registration by the institution, OAuth2 per student, a database with encryption at rest, validating the Gemini mentor against the real API with a paid institutional key, a formal privacy notice and an external security review. Chapter 9 has the full list.

Does FARO replace Canvas? No. Canvas remains the academic source of truth. FARO reads from Canvas and adds a companionship layer. *We don't replace Canvas: we make it adaptive.*

9. Pre-pilot checklist

Tick every item before connecting FARO to a Canvas with real students. Items marked ⚙ are verified by the technical team; items marked 🏛 by the institution.

A. Before starting the backend

- ☐ ⚙ `server/.env` exists, is **not** in git (`git status` does not list it) and `CANVAS_API_URL` begins with `https://`.
- ☐ ⚙ The token belongs to a pilot **student account**, not an instructor or administrator.
- ☐ ⚙ The token has an **expiry date** (recommended: 30 days).
- ☐ ⚙ `FARO_HOST=127.0.0.1` unless TLS sits in front.
- ☐ ⚙ `FARO_ALLOWED_ORIGINS` contains only the real origins (the loaded extension's exact id, the dev server if applicable).
- ☐ ⚙ `FARO_STRICT_ORIGINS=true` in any deployment that is not a developer's machine: the backend then stops accepting any `chrome-extension://` origin (chapter 3.6).
- ☐ ⚙ If the Gemini mentor will be used: `GEMINI_API_KEY` belongs to a Google Cloud project **with active billing** (Paid Services), **never** the free tier (chapter 7.8).
- ☐ ⚙ `GEMINI_MODEL` has been checked against `ai.google.dev/gemini-api/docs/models` (default `gemini-3.5-flash`; model names change).
- ☐ ⚙ `FARO_MENTOR_PER_MINUTE` has an agreed value (default 20 model calls per client per minute).
- ☐ ⚙ `npm test` in `server/` passes all 38 checks.
- ☐ ⚙ `curl http://127.0.0.1:3000/health` answers with the Canvas host and `mentor:` `{provider, model}` (or `null` without a key), and does **not** contain the token or the key.
- ☐ ⚙ The Gemini mentor has been tried against the real API with the pilot key: a reply arrives with the violet ring and **without** the note "Gemini could not answer...". (In development it was only verified against a fake Gemini.)

B. Before loading the extension

- ☐ ⚙ It was built with `VITE_CANVAS_MODE=http` and `VITE_FARO_API_URL` pointing at the backend.
- ☐ ⚙ `VITE_MENTOR_MODE=local` unless the institution approved the AI mentor; in that case `auto` (the default) or `gemini`, with an approved paid key on the server. The FARO Mentor footer says the right thing ("Answers by Gemini, through FARO's server..." or "Local mentor: no AI model...").
- ☐ ⚙ Neither the root `.env` nor `.env.local` contains a Gemini key.
- ☐ ⚙ `public/manifest.json` does **not** contain `instructure.com` in `host_permissions`.
- ☐ ⚙ In *Profile* → *Canvas* the badge says *Live* and the log shows status `200` on all four paths.
- ☐ ⚙ If the note "your account cannot read analytics" appears, it was decided with the institution whether to enable the permission or accept the submission-based fallback.

C. With the institution

- ☐ ⚙ Which law applies has been confirmed (LFPDPPP for a private institution; Ley General for a public one).
- ☐ ⚙ A privacy notice exists (simplified + full), reviewed by legal, mentioning: the six Canvas paths, the data FARO creates, the "life happened" state, the Gemini mentor if enabled (what is sent to Google, chapter 7.6), and retention.
- ☐ ⚙ Every participant signed the pilot consent (chapter 7.4) and is 18 or over (a requirement of the Gemini API terms).
- ☐ ⚙ It was agreed how ARCO rights are exercised during the pilot and who responds.
- ☐ ⚙ It was agreed which pilot metrics are delivered and in what form (aggregated, with pseudonymous ids).
- ☐ ⚙ The institution's security team reviewed this book and the `server/` code.
- ☐ ⚙ The pilot end date and the procedure for revoking tokens and deleting data at the end were defined.

D. When the pilot ends

- ☐ ⚙ All pilot tokens revoked in Canvas.
- ☐ ⚙ The pilot's Gemini key revoked or rotated in Google.
- ☐ ⚙ `server/.env` deleted from the backend machine.
- ☐ ⚙ Participants told how to delete local state (*Progress* → *Reset* or *uninstall*).
- ☐ ⚙ Metrics delivered; raw data destroyed or handed to the institution as agreed.

E. To move to production (outside the pilot's scope)

- ☐ Registration of FARO as an account-level LTI 1.3 tool in Canvas, with exactly the scopes in `requiredScopes` and without `score` or `lineitem`.
- ☐ Implementation of `/lti/login`, `/lti/launch` and `/.well-known/jwks.json` per the specification in `server/src/routes/lti.ts`.
- ☐ OAuth2 per student; removal of the shared token.
- ☐ PostgreSQL with encryption at rest; automatic retention policy.
- ☐ Legal review of the Gemini API terms in force (or those of whichever provider is chosen) and of the institution's agreement with Google.
- ☐ External security review.